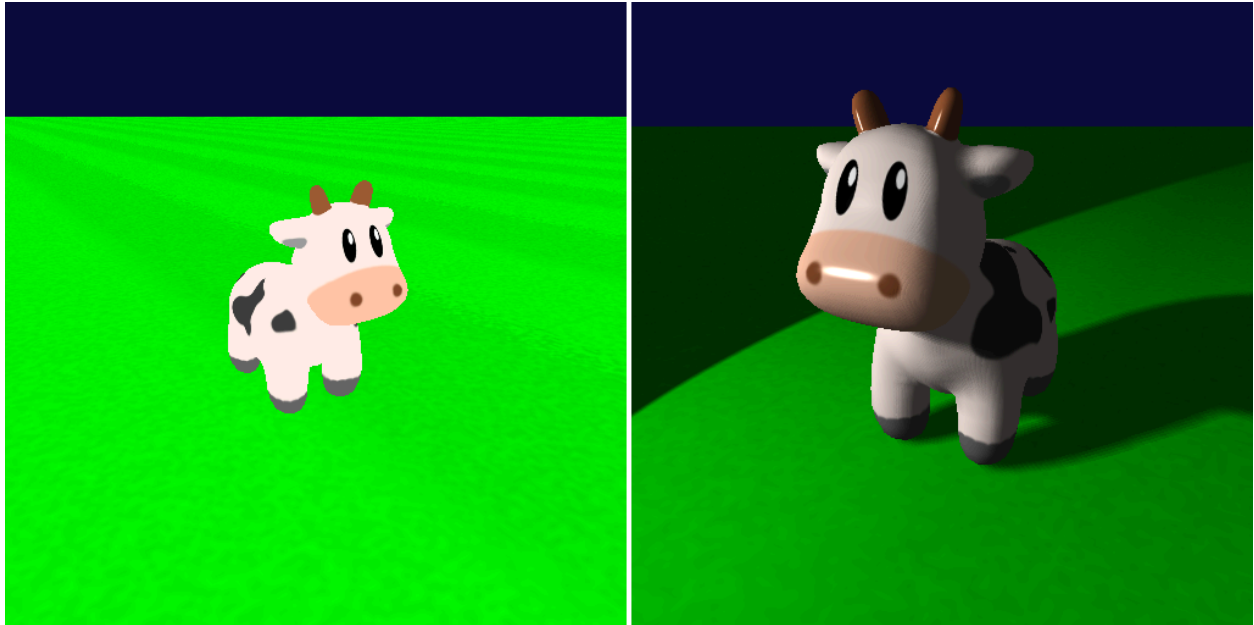


Project 4: Shadows, Spotlights, & Spot

Name: Dane Christensen

Date: 8/4/2026



For this project you will implement shadow mapping using multiple passes by means of WebGL's Framebuffer object.

Although you are welcome to use your Software Renderer to implement this project, you are recommended to use the WebGL2 API.

- **Starting File**

You will notice your output deliverable is similar to another in class demo that exhibits "Cell Shading" of Spot the cow. You are recommended to use and extend this environment to build out the cell shading effect. You will need to:

- Create a new file in the "Effects" JS folder appropriately named for this project
- Add a new file in the "Cameras" folder that will model a "LookAt" camera. You will need to implement the math to control the spotlight's movement and orientation through a position/direction vector pair of control inputs (FOV and near/far planes are other inputs required as well).
- Write your own shaders. Whatever shaders you use should be detailed/setup in the "shaders/3D" part of the file tree.

- **Camera Look At Function**

Think back to our earlier units on Vectors and Matrices. Everything you need to implement a "look-at" style camera exists in that earlier work. Although there are different ways to go about this, using the cross product to compute missing vectors and using your vectors as the basis for your light's "view" transformation will be essential to an optimal solution. Remember that in this context, the camera does not move around the world, rather, the world revolves around us. If we were to move and then rotate our camera, we would move the world the opposite way and rotate the world around us in the opposite direction.

- **Shadow Mapping**

Your shadow mapping effect, like the cell shading effect before it should show two views of the scene. Except for any cameras and view/projection matrices, your new "JS" class to capture this effect should manage its own state.

The first view should give a base "albedo" color of the scene from the point of view of the light source. The "albedo" shader from the reference example will help you in completing this part of the challenge. Although both color and depth values are being generated, the former are for visual intuition/grading benefit and the latter is more essential to implementing shadow mapping.

The second view should be the polished rendering of spot the cow. Use the Blinn-Phong shading model from Project #3 here. Your light source should be nearby "Spot" and clearly indicate the shadow mapping capabilities of your render. Although we usually think of being in shadow as a boolean value, here it can be a coefficient (let's call it "illuminated") in the range $[0, +1]$ should you take many samples and find some are light visible, others not. You can say that the ambient term is a constant such as "0.1" based on the albedo color of each primitive and then multiply the remaining diffuse/specular components by the "illuminated" value.

You will need to pass a reference to a "depth texture" as used in the first half of your render. You will also need to interpolate a world space position between each triangle's vertices, so the fragment shader can know each fragment's world space position. With that, you can calculate what depth value would be calculated for that fragment in the light's rendering pass based on the view/projection matrices used there. If the corresponding value in the depth buffer closely matches the one you calculate, then you can assume that the fragment is seen by the light source.

Make sure that you understand how the matrices transform a world space position to NDC and then to the viewport transform (for us, this last transform will directly map to UV coordinates). You may want to manually multiply matrices/vectors together in JS with your browser's IDE to check that your understanding is complete. Without such a mapping, you will be unable to relate your shadow map to your rendering of Spot.

You will want to take multiple samples of the visible/not visible test. About 25 tests, spread across adjacent X/Y pixels in the depth buffer should suffice. Even with all this filtering, you will see some pixelated artifacts. It is important to tolerate a certain amount of precision error. The range of that error is a fine tuning detail I leave to you to choose.

- **Last thoughts:**

A. This project is a challenging one for its WebGL complexity. Save your progress often and start out simple while working on the two features described above.

B. When nothing is rendering, consider tracing the math through by hand in the JS console. The interactive session the debugger affords should make it easy to see how points transform/relate to each other.

C. When debugging your GLSL shaders, I recommend that you use the colors as a debugging aid. If a condition ("if" statement) is in your code, you can rewrite it so that a particular color is sent to the screen. Use this to make educated guesses/tests of what is happening within this pipeline stage.